

Cliente SGA Rest Web Service



 **Guía:**  **Crear el cliente RESTful para consumir los Web Services REST de SGA**

 **Objetivo de esta sección:**

Crear un nuevo proyecto Maven en NetBeans que actuará como **cliente RESTful**, y configurar correctamente sus dependencias para comenzar a hacer peticiones al Web Service.

-  Lectura de una persona
-  Listado de todas las personas
-  Inserción de una nueva persona
-  Modificación de una persona existente
-  Eliminación de una persona

Paso 1: Crear un nuevo proyecto

Abrimos NetBeans y seguimos estos pasos:

1. Clic en **File > New Project...**
 2. Seleccionamos la categoría **Java with Maven > Java Application**
 3. Escribimos el nombre del proyecto: **ClienteSgaRestWs**
 4. Ubicamos el proyecto en la carpeta: **C:\Cursos\Java\JakartaEE**
 5. El **GroupId** es: **sga**
 6. Dejamos el paquete vacío
 7. Clic en **Finish**
-

✿ Paso 2: Modificar el archivo `pom.xml`

Agregamos las dependencias necesarias de Jakarta EE, Jersey, entre otras.

El archivo final queda así:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>sga</groupId>
  <artifactId>ClienteSgaRestWs</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencies>
    <dependency>
      <groupId>jakarta.platform</groupId>
      <artifactId>jakarta.jakartaee-api</artifactId>
      <version>11.0.0-M1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>org.glassfish.jersey.core</groupId>
      <artifactId>jersey-client</artifactId>
      <version>3.1.10</version>
    </dependency>
    <dependency>
      <groupId>org.glassfish.jersey.media</groupId>
      <artifactId>jersey-media-jaxb</artifactId>
      <version>3.1.10</version>
    </dependency>
  </dependencies>
</project>
```

```

<dependency>
  <groupId>org.glassfish.jersey.inject</groupId>
  <artifactId>jersey-hk2</artifactId>
  <version>3.1.10</version>
</dependency>
<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
  <version>4.0.2</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
  <version>4.0.5</version>
</dependency>

</dependencies>
</project>

```

💡 Las librerías del proyecto Jersey te permiten:

- Hacer peticiones HTTP REST (cliente Jersey)
- Convertir objetos Java a XML/JSON y viceversa (JAXB)
- Manejar inyecciones de dependencias internas de Jersey (HK2)

📥 Paso 3: Descargar las dependencias

Da clic derecho en el proyecto > **Clean and Build** para descargar todas las dependencias de Maven. Asegúrate de que no se generen errores y todo esté listo ✅.

Hasta aquí hemos creado la base del cliente RESTful 🏗️.

En la siguiente parte, te mostraré cómo crear la clase `Persona.java` para enviar y recibir objetos desde el servicio web 🧑 ➡️ 🌐. ¡Vamos con eso enseguida! 😊

🧑 Paso 4: Crear la clase de dominio `Persona.java`

Ahora que ya tenemos configurado el proyecto, vamos a crear una clase sencilla llamada `Persona` para poder enviar y recibir datos desde nuestro Web Service. Esta clase es equivalente a la entidad en el servidor, pero del lado del cliente.

Crear el paquete y la clase

1. Clic derecho en **Source Packages**
2. Selecciona **New > Java Class**
3. Nombre de la clase: `Persona`
4. Paquete: `dominio`
5. Clic en **Finish**

Escribir el código de la clase `Persona`

Agrega el siguiente contenido a la clase `Persona.java`:

```
package dominio;

import jakarta.xml.bind.annotation.XmlRootElement;

@XmlRootElement
public class Persona {

    private int idPersona;
    private String nombre;
    private String apellido;
    private String email;
    private String telefono;

    public Persona(){}

    public Persona(int idPersona){
        this.idPersona = idPersona;
    }

    public int getIdPersona() {
        return idPersona;
    }

    public void setIdPersona(int idPersona) {
        this.idPersona = idPersona;
    }

    public String getNombre() {
```

```
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido +
            ", email=" + email + ", telefono=" + telefono + '}';
    }
}
```

Explicación breve

- `@XmlRootElement`: permite que esta clase pueda ser convertida automáticamente a XML (o desde XML) al hacer peticiones con Jersey.

- Tiene un constructor vacío y otro con la clave primaria `idPersona`.
 - Incluye todos los getters, setters y un método `toString()` para mostrar los datos fácilmente.
-

🎉 ¡Listo! Ya tenemos nuestro objeto de dominio del lado cliente.

Continuemos con la guía creando y explicando paso a paso la clase `TestPersonaServiceRS` que nos permitirá **probar todos los métodos del Web Service RESTful**: listar, buscar, agregar, modificar y eliminar personas. ¡Todo desde el cliente! 🧪 🌐

🧪 Paso 5: Crear la clase de prueba `TestPersonaServiceRS`

Esta clase actuará como nuestro cliente REST. Vamos a usar `javax.ws.rs.client` para conectarnos al Web Service, enviar solicitudes y procesar las respuestas.

📦 Crear la clase

1. Clic derecho sobre **Source Packages**
 2. Selecciona **New > Java Class**
 3. Nombre de la clase: `TestPersonaServiceRS`
 4. Paquete: `test`
 5. Clic en **Finish**
-

💡 Antes del código...

Vamos a declarar variables globales para reutilizar durante las peticiones REST:

```
package test;

import domain.Persona;
import java.util.List;
import jakarta.ws.rs.client.*;
import jakarta.ws.rs.core.*;

public class TestPersonaServiceRS {

    //Variables que vamos a utilizar
    private static final String URL_BASE = "http://localhost:8080/sga-jee-web/webservice";
```

```
private static Client cliente;
private static WebTarget webTarget;
private static Persona persona;
private static List<Persona> personas;
private static Invocation.Builder invocationBuilder;
private static Response response;

public static void main(String[] args) {
    cliente = ClientBuilder.newClient();
```

Leer una persona por ID (GET con path)

```
//Leer una persona (metodo get)
webTarget = cliente.target(URL_BASE).path("/personas");
//Proporcionamos un idPersona valido
persona = webTarget.path("/1").request(MediaType.APPLICATION_XML).get(Persona.class);
System.out.println("Persona recuperada:" + persona);
```

Explicación:

- `.target(URL_BASE)`: apunta a la raíz del servicio
 - `.path("/personas/1")`: llama al servicio GET para obtener la persona con `id = 1`
 - `.request(XML).get(...)`: indica que esperamos la respuesta en XML y que la convertiremos a objeto `Persona`
-

Leer todas las personas (GET sin path)

```
//Leer todas las personas (metodo get con readEntity de tipo List<>
response = webTarget.request(MediaType.APPLICATION_XML).get();
personas = response.readEntity(new GenericType<List<Persona>>({}));
System.out.println("\nPersonas recuperadas");
personas.forEach(System.out::println);
```

Explicación:

- No se especifica ID, por lo que llama al método `listarPersonas`.
 - Usamos `GenericType<List<Persona>>` para convertir la respuesta XML a una lista de objetos Java.
 - Imprimimos la lista con `forEach`.
-

+ Agregar nueva persona (POST)

```
//Agregar una persona (metodo post)
Persona nuevaPersona = new Persona();
nuevaPersona.setNombre("Carlos");
nuevaPersona.setApellido("Miranda");
nuevaPersona.setEmail("cmiranda3@mail.com");
nuevaPersona.setTelefono("99887700");

invocationBuilder = webTarget.request(MediaType.APPLICATION_XML);
response = invocationBuilder.post(Entity.entity(nuevaPersona, MediaType.APPLICATION_XML));
System.out.println("");
System.out.println(response.getStatus());
//Recuperamos la personas recién agregada para despues modificarla y al final eliminarla
Persona personaRecuperada = response.readEntity(Persona.class);
System.out.println("Persona agregada:" + personaRecuperada);
```

✖ Explicación:

- Creamos una persona nueva con valores ficticios.
 - Hacemos POST al path /personas.
 - Usamos Entity.entity(..., MediaType.XML) para enviar el objeto serializado.
 - Imprimimos el estado de la respuesta y los datos retornados.
-

✎ Modificar persona (PUT)

```
//Modificar la persona (metodo put)
//persona recuperada anteriormente
Persona personaModificar = personaRecuperada;
personaModificar.setApellido("Ramirez");
String pathId = "/" + personaModificar.getIdPersona();
invocationBuilder = webTarget.path(pathId).request(MediaType.APPLICATION_XML);
response = invocationBuilder.put(Entity.entity(personaModificar, MediaType.APPLICATION_XML));

System.out.println("");
System.out.println("response:" + response.getStatus());
System.out.println("Persona modifica:" + response.readEntity(Persona.class));
```

✖ Explicación:

- Modificamos el apellido.
- El PUT requiere pasar el id como path (/personas/{id}).
- Enviamos el nuevo objeto con Entity.entity(...).

🗑 Eliminar persona (DELETE)

```
//Eliminar una persona
//persona recuperada anteriormente
Persona personaEliminar = personaRecuperada;
String pathEliminarId = "/" + personaEliminar.getIdPersona();
invocationBuilder = webTarget.path(pathEliminarId).request(MediaType.APPLICATION_XML);
response = invocationBuilder.delete();
System.out.println("");
System.out.println("response:" + response.getStatus());
System.out.println("Persona Eliminada" + personaEliminar);
```

✦ Explicación:

- Llamamos a DELETE sobre /personas/{id}.
- Mostramos el código de respuesta HTTP (200 si fue exitosa).

Ya con esto tenemos cubiertos los cinco métodos básicos de un **cliente REST completo**:

Acción	Método HTTP	Descripción
Listar personas	GET	/personas
Buscar por ID	GET	/personas/{id}
Agregar persona	POST	/personas
Modificar persona	PUT	/personas/{id}
Eliminar persona	DELETE	/personas/{id}

✦ Código completo de la clase TestPersonaServiceRS.java:

```
package test;

import domain.Persona;
import java.util.List;
import jakarta.ws.rs.client.*;
import jakarta.ws.rs.core.*;

public class TestPersonaServiceRS {

    //Variables que vamos a utilizar
    private static final String URL_BASE = "http://localhost:8080/sga-jee-web/webservice";
    private static Client cliente;
    private static WebTarget webTarget;
```

```
private static Persona persona;
private static List<Persona> personas;
private static Invocation.Builder invocationBuilder;
private static Response response;

public static void main(String[] args) {
    cliente = ClientBuilder.newClient();

    //Leer una persona (metodo get)
    webTarget = cliente.target(URL_BASE).path("/personas");
    //Proporcionamos un idPersona valido
    persona = webTarget.path("/1").request(MediaType.APPLICATION_XML).get(Persona.class);
    System.out.println("Persona recuperada:" + persona);

    //Leer todas las personas (metodo get con readEntity de tipo List<>
    response = webTarget.request(MediaType.APPLICATION_XML).get();
    personas = response.readEntity(new GenericType<List<Persona>>({}));
    System.out.println("\nPersonas recuperadas");
    personas.forEach(System.out::println);

    //Agregar una persona (metodo post)
    Persona nuevaPersona = new Persona();
    nuevaPersona.setNombre("Carlos");
    nuevaPersona.setApellido("Miranda");
    nuevaPersona.setEmail("cmiranda3@mail.com");
    nuevaPersona.setTelefono("99887700");

    invocationBuilder = webTarget.request(MediaType.APPLICATION_XML);
    response = invocationBuilder.post(Entity.entity(nuevaPersona, MediaType.APPLICATION_XML));
    System.out.println("");
    System.out.println(response.getStatus());
    //Recuperamos la personas recién agregada para después modificarla y al final eliminarla
    Persona personaRecuperada = response.readEntity(Persona.class);
    System.out.println("Persona agregada:" + personaRecuperada);

    //Modificar la persona (metodo put)
    //persona recuperada anteriormente
    Persona personaModificar = personaRecuperada;
    personaModificar.setApellido("Ramirez");
    String pathId = "/" + personaModificar.getIdPersona();
    invocationBuilder = webTarget.path(pathId).request(MediaType.APPLICATION_XML);
    response = invocationBuilder.put(Entity.entity(personaModificar, MediaType.APPLICATION_XML));

    System.out.println("");
    System.out.println("response:" + response.getStatus());
}
```

```

        System.out.println("Persona modifica:" + response.readEntity(Persona.class));

        //Eliminar una persona
        //persona recuperada anteriormente
        Persona personaEliminar = personaRecuperada;
        String pathEliminarId = "/" + personaEliminar.getIdPersona();
        invocationBuilder = webTarget.path(pathEliminarId).request(MediaType.APPLICATION_XML);
        response = invocationBuilder.delete();
        System.out.println("");
        System.out.println("response:" + response.getStatus());
        System.out.println("Persona Eliminada" + personaEliminar);
    }
}

```



Paso 6: Ejecutar la clase de prueba

1. Asegúrate de que el proyecto `sga-jee-web` esté **desplegado y funcionando en GlassFish**.
2. Ejecuta la clase `TestPersonaServiceRS` con clic derecho → **Run File**
3. Debes ver una salida similar a:

```

Persona recuperada:Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}

```

```

Personas recuperadas

```

```

Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}

```

```

Persona{idPersona=2, nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}

```

```

Persona{idPersona=3, nombre=Maria, apellido=Esparza, email=mesparza@mail.com, telefono=55112233}

```

```

Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}

```

```

Response: 200

```

```

Persona agregada:Persona{idPersona=29, nombre=Carlos, apellido=Miranda, email=cmiranda3@mail.com, telefono=99887700}

```

```

response:200

```

```

Persona modifica:Persona{idPersona=29, nombre=Carlos, apellido=Ramirez, email=cmiranda3@mail.com, telefono=99887700}

```

```

response:200

```

```
Persona EliminadaPersona{idPersona=29, nombre=Carlos, apellido=Ramirez, email=cmiranda3@mail.com, telefono=99887700}
```

✅ Conclusión del ejercicio

🎉 ¡Felicidades! Has construido un **cliente RESTful completo** que se comunica con tu servicio web de SGA utilizando Jersey y el API de JAX-RS.

🔍 En este ejercicio aprendiste:

- Cómo consumir un servicio RESTful con Java y Jersey
- Cómo manejar diferentes tipos de peticiones HTTP: GET, POST, PUT, DELETE
- Cómo trabajar con clases Java simples para serializar y deserializar XML/JSON
- Cómo hacer pruebas controladas agregando, modificando y eliminando registros

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)